Algorithms and Data Structures I.
Endterm Test Sample

Note: Do not write on this paper! Hand back with your answers!
Write your *name, neptun code, test id (e.g. A, B) and practice course code* on every paper.
Only using empty paper and pen is allowed! Cheating will result in a failed test!

Task 1. (10 points):
a) Construct a **Binary Search Tree** with the insertion of the following elements:

$$\langle 19, 17, 30, 10, 21, 12, 42, 25, 33, 34 \rangle$$

Demonstrate the final tree of the data structure after the final insertion.

b) Write down the data structure's keys in *preorder* traversal.

c) Delete the keys 12 and then 30 from the tree, maintaining the *BST* structure property. Demonstrate the tree after each of the deletions separately.

Task 2. (10 points):
Given an **open-addressed hash table** of size 8 using a quadratic probe sequence. Use the hash function: $h(k, n) = (k + \frac{n+n^2}{2}) \bmod 8$. Demonstrate how the table changes with every operation!

Operation	k	$h_1(k)$	Probe sequence	Result	0	1	2	3	4	5	6	7
Ins	13											
Ins	20											
Ins	31											
Ins	87											
Ins	12											
Del	31											
Search	12											
Search	15											
Ins	4											
Del	10											
Ins	35											
Ins	10											

Task 3. (15 points):
Sort the array with **HeapSort**.

$$\langle 4, 1, 3, 2, 18, 9, 10, 14, 12, 7 \rangle$$

a) Demonstrate the BuildMaxHeap() algorithm.

b) Demonstrate the first 2 iteration of the Sorting() algorithm. Note the *swap* and the comparisons during *sink()* and the resulting tree after each iteration.

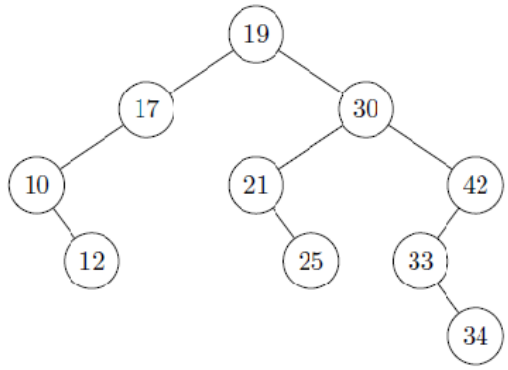
Task 4. (10 points):
Given a **Binary tree** (t), provide an algorithm that returns the number of nodes having a single child in the first **k** level.

Task 5. (15 points):
You are given a sequence of characters to read on input using the *read(x)* procedure.
Design an algorithm that uses a **Queue**. Write on the output a boolean answer Which represents if the first half of the subsequences is the same as the second half. Present your algorithm in structogram format.

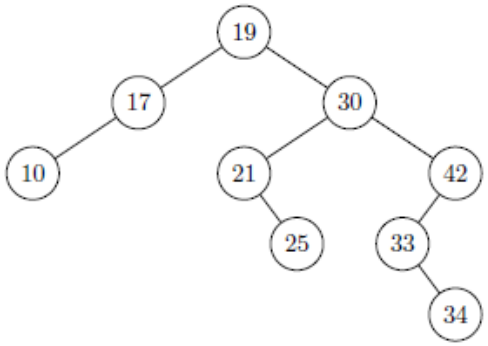
Examples:	aa	→	TRUE
	appleapple	→	TRUE
	applebanana	→	FALSE

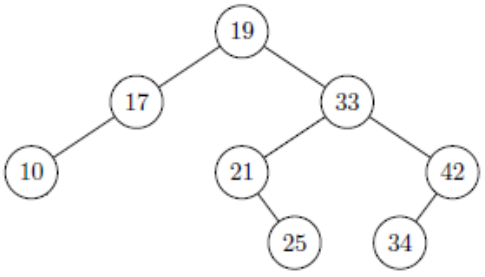
Algorithms and data structures 1
Sample Solutions

Solution 1.
a)



b)
$\langle 19, 17, 10, 12, 30, 21, 25, 42, 33, 34 \rangle$
c)





Solution 2.

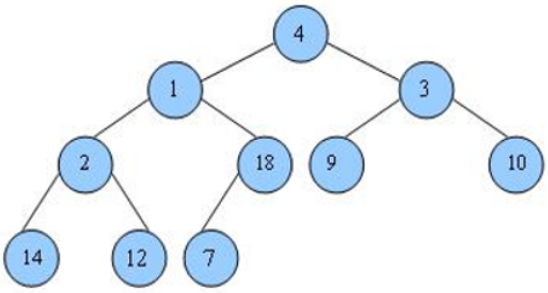
$m = 8$, $h : \mathbb{N} \to 0..7$, $h(k, n) = (k + \frac{n+n^2}{2}) \bmod 8$.

	k	$h_1(k)$	Actual probing sequence	0	1	2	3	4	5	6	7
ins	13	5	5_E ✓						13		
ins	20	4	4_E ✓					20	13		
ins	31	7	7_E ✓					20	13		31
ins	87	7	$7, 0_E$ ✓	87				20	13		31
ins	12	4	$4, 5, 7, 2_E$ ✓	87		12		20	13		31
del	31	7	7_{31} ✓	87		12		20	13		D
search	12	4	$4, 5, 7_D, 2_{12}$ ✓	87		12		20	13		D
search	15	7	$7_D, 0, 2, 5, 1_E$ ✗	87		12		20	13		D
ins	4	4	$4, 5, 7_D, 2, 6_E$ ✓	87		12		20	13		4
del	10	2	$2, 3_E$ ✗	87		12		20	13		4
ins	35	3	3_E ✓	87		12	35	20	13		4
ins	10	2	$2, 3, 5, 0, 4, 1_E$ ✓	87	10	12	35	20	13		4

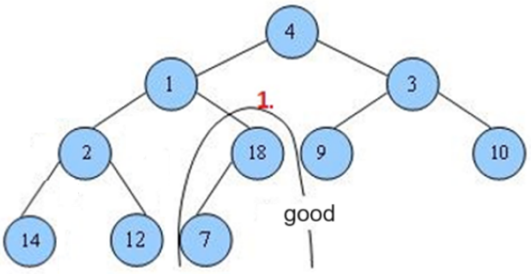
Solution 3.

a)

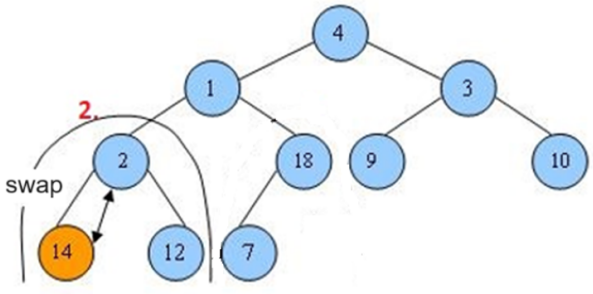
Initial step:



First step:

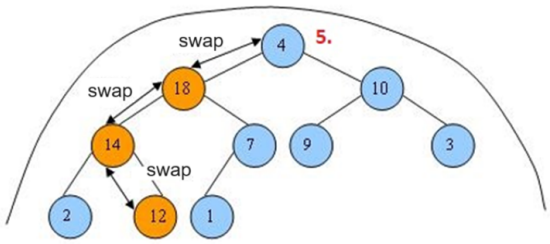


Second step:

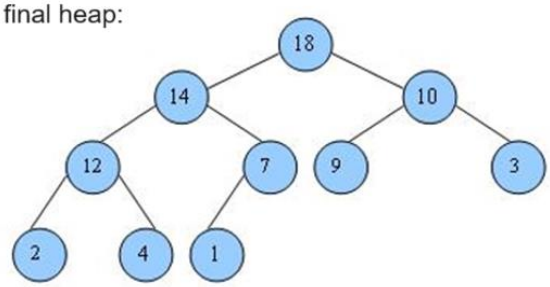


and so on with the steps...

Last step:

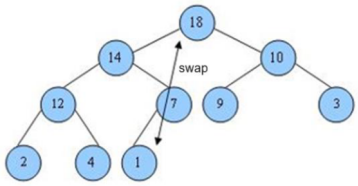


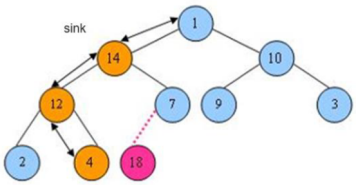
final heap:

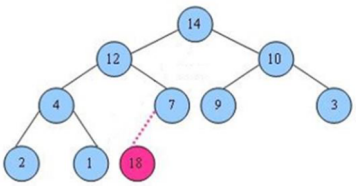


b)

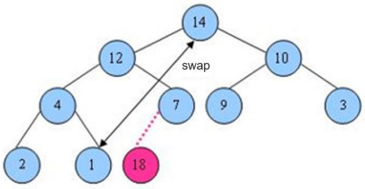
First step:

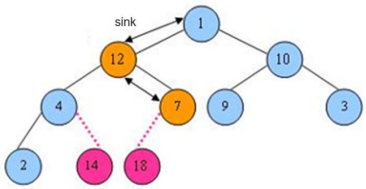


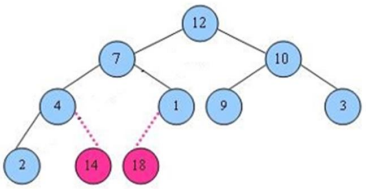




Second step:







Solution 4.

SingleChild (t:Bintree, k: N) : N

t = 0 OR k < 0		
return 0	c1:= SingleChild(t->left,k-1)	
	c2:=SingleChild(t->right,k-1)	
	(t->left≠0 & t->right=0) OR (t->left=0 & t->right≠0)	
	return c1+c2+1	return c1+c2

Solution 5.

Doubled words (): B